

# **OOP Lab Project Report**

## **File-Based Chat Application**

---

Submitted in Partial Fulfillment of the Requirements for  
**OOP Laboratory Course**

Developed in C++17  
IDE: Visual Studio Code / MinGW  
Platform: Windows

**Department of Computer Science**

# 1. Introduction

## 1.1 Project Overview

The File-Based Chat Application is a console-based, menu-driven messaging system developed in C++17. It enables registered users to exchange private messages through a secure, file-backed storage system. The application supports two types of users — Admin and Member — each with role-specific capabilities accessed through a color-coded, interactive terminal interface.

Users can register accounts, log in securely, exchange private messages (stored with Caesar-cipher encryption), view full chat histories, and search messages by keyword or sender. Admins additionally have the power to view all users, remove accounts, and unlock locked accounts. All data — users, messages, and application logs — persists to disk via plain-text files managed through the FileManager class.

## 1.2 Problem Statement

Modern communication systems rely on complex infrastructure such as databases and network sockets. However, there is a clear need for lightweight, self-contained messaging applications that operate offline, require no external services, and demonstrate core software engineering principles. This project addresses that need by building a fully functional chat application using only C++ standard library facilities, file I/O, and OOP design patterns.

Real-world challenges solved by this application include:

- Persistent user and message storage without a database engine
- Secure credential storage using encryption
- Role-based access control (Admin vs. Member)
- Account lockout after repeated failed login attempts
- Keyword and sender-based message search across all chats

## 1.3 Objectives

- **Apply core OOP principles — encapsulation, inheritance, polymorphism, and abstraction — in a real working C++ application**
  - **Implement persistent file-based storage using `fstream` for all user data, messages, and logs**
  - **Demonstrate modern C++ memory management using smart pointers (`std::unique_ptr`) to eliminate memory leaks**
  - **Utilize C++ Standard Template Library (STL) containers (`vector`, `unordered_map`, `set`) and algorithms for efficient data handling**
-

## 2. System Features

### 2.1 User Registration

New users register by providing a username, password, account type (admin or member), and a security question with an answer. The system validates that the username is unique using `std::find_if` over the users vector. Duplicate registrations are rejected with a descriptive error. Passwords and security answers are stored encrypted on disk.

### 2.2 User Login and Account Security

Users log in with a username and password. The `User::login()` method tracks consecutive failed attempts using `loginAttempts_`. After three failures (`kMaxLoginAttempts = 3`), the account is automatically locked and cannot be accessed until an Admin calls `User::unlock()`. A Forgot Password feature allows users to reset their password by correctly answering their security question.

### 2.3 Private Messaging

Members can initiate a private chat with any other registered user. Messages are created as `EncryptedMessage` objects using a Caesar cipher before being stored in memory and on disk. The chat session menu offers options to send new messages, view the full chat history, and return to the dashboard.

### 2.4 Viewing Chat History

The view history feature displays all messages in a private chat in chronological order. Each `EncryptedMessage` is decrypted on-the-fly during display using `EncryptedMessage::display()`, ensuring that stored data remains encrypted at all times. After viewing, all messages are marked as read via `PrivateChat::markAsRead()`.

### 2.5 Message Search

Users can search all their private conversations using `SearchEngine`. Two search modes are available: keyword-only search (`SearchEngine::search` with one query parameter) and keyword-plus-sender search (the overloaded variant with two parameters). Results are printed to the console with sender name and timestamp.

### 2.6 Admin Panel

Admin users see a separate menu upon login. They can: remove a user account (and automatically delete associated chat files), unlock a locked account, or view a complete list of all registered users. These administrative operations are guarded by the `canModerate()` virtual method — only Admin returns true.

### 2.7 Menu-Driven Interface

The entire application is driven by numbered menus displayed using `printHeader()` and `printMenu()` utility functions. Console output uses ANSI escape codes for color highlighting — blue for menus, green for success messages, red for errors, and yellow for prompts — enhancing usability in supported terminals.

### 2.8 File-Based Persistent Storage

All data is stored in a structured directory:

- data/users.txt — stores user records in CSV format (type, username, encrypted password, security question, encrypted answer)
  - data/private\_chats/<id>.txt — stores each chat session as a flat file with one message per line
  - data/logs/app.log — append-only log file managed by the Logger<string> template class
-

### 3. System Specifications

**Development Environment:** Visual Studio Code with MinGW-w64 / GCC compiler toolchain

**Compiler Standard:** C++17 (required for `std::filesystem`, `std::make_unique`, and structured bindings)

**Operating System:** Windows 10/11 (the application uses `system("cls")` for screen clearing; ANSI codes work in Windows Terminal)

**Build Method:** Manual g++ compilation or makefile; no external third-party libraries required

**File I/O Library:** Standard C++ `<fstream>`, `<sstream>`, `<filesystem>`

**STL Version:** GNU libstdc++ (bundled with MinGW)

---

## 4. OOP Features and Technical Implementation

### 4.1 Classes and Objects

The project is composed of nine well-defined classes, each with a single responsibility:

- User (abstract) — base class for all users; holds authentication state and block list
- Admin : User — concrete class for privileged users; overrides canModerate() to return true
- Member : User — concrete class for regular users; overrides canModerate() to return false
- Message (abstract) — base class for all message types; holds sender, content, timestamp, and read status
- TextMessage : Message — stores and displays plain text messages
- EncryptedMessage : Message — encrypts content on construction; decrypts on display
- PrivateChat — owns a vector<unique\_ptr<Message>>; manages a two-user conversation
- FileManager — handles all disk I/O for users and chats
- SearchEngine — searches messages by keyword and/or sender
- EncryptionManager — stateless Caesar-cipher encrypt/decrypt utility
- Logger<T> — generic template class for timestamped file logging

### 4.2 Encapsulation

All class data members are declared private (prefixed with underscore by convention). Access is provided exclusively through public getter and setter methods. Setters include validation logic:

```
// User.cpp
void User::setPassword(const std::string& password) {
    if (password.empty())
        throw std::invalid_argument("Password cannot be empty.");
    password_ = password;
    loginAttempts_ = 0;
}
```

The static constant kMaxLoginAttempts is encapsulated inside the User class, preventing external code from modifying the lockout threshold.

### 4.3 Constructors and Destructors

Every class defines explicit constructors that initialize all member variables and validate inputs. The base class constructors are called via initializer lists in derived classes:

```
// Admin.cpp
Admin::Admin(const std::string& username, const std::string& password)
    : User(username, password) {}
```

User and Message both declare pure virtual destructors. This is a mandatory C++ requirement for abstract base classes — it ensures the correct derived-class destructor is called when a base-class pointer is deleted. The definition must still exist:

```
// User.cpp
User::~~User() {} // Definition required for linkage
```

Smart pointer containers (e.g., vector<unique\_ptr<User>>) automatically call destructors without any explicit delete calls in the code.

## 4.4 Inheritance

The project implements two-level inheritance hierarchies:

Hierarchy 1 — User system:

- User (abstract base) ← Admin (concrete) and Member (concrete)

Hierarchy 2 — Message system:

- Message (abstract base) ← TextMessage (concrete) and EncryptedMessage (concrete)

Both hierarchies allow the application to store heterogeneous collections (e.g., `vector<unique_ptr<User>>` holds both Admin and Member objects) and treat them uniformly through base-class pointers. Derived classes selectively override only the virtual methods they need to specialize.

## 4.5 Polymorphism

### 4.5.1 Static Polymorphism (Compile-Time)

Function overloading is demonstrated in `SearchEngine`, which provides two `search()` methods sharing the same name but with different parameter signatures:

```
// SearchEngine.h
// Search by keyword only:
std::vector<const Message*> search(const vector<unique_ptr<Message>>& messages,
                                   const string& keyword) const;

// Search by keyword AND sender:
std::vector<const Message*> search(const vector<unique_ptr<Message>>& messages,
                                   const string& keyword,
                                   const string& sender) const;
```

Operator overloading is used in both User and Message classes. The stream insertion operator (`<<`) is overloaded to allow objects to be printed directly:

```
// User.cpp
std::ostream& operator<<(std::ostream& os, const User& user) {
    os << "Username: " << user.username_
    << " | Status: " << (user.isOnline_ ? "Online" : "Offline");
    return os;
}
```

A type conversion operator is also defined in User, enabling implicit conversion of a User object to its username string:

```
explicit operator std::string() const { return username_; }
```

### 4.5.2 Dynamic Polymorphism (Runtime)

Virtual dispatch is the core mechanism that enables role-specific behavior. Both User and Message declare pure virtual methods:

```
// User.h — pure virtual methods
virtual void displayProfile() const = 0;
virtual bool canModerate() const = 0;

// Message.h — pure virtual methods
virtual void display() const = 0;
virtual std::string getType() const = 0;
virtual std::string getContent() const { return content_; }
```

At runtime, when `main.cpp` calls `currentUser->canModerate()`, the vtable dispatches to `Admin::canModerate()` (returns true) or `Member::canModerate()` (returns false). Similarly,

msg->display() dispatches to either TextMessage::display() or EncryptedMessage::display() depending on the actual object type. This is the foundation of the admin/member menu split and the transparent encryption/decryption feature.

## 4.6 Abstraction

Implementation details are hidden behind clean interfaces in header files (.h). The caller never needs to know how EncryptionManager::encrypt() shifts characters — they simply call the method. Similarly, FileManager hides the entire filesystem traversal logic; the rest of the application only calls fm.saveUsers() or fm.loadPrivateChat().

Abstract base classes (User, Message) enforce a contract: any concrete subclass must implement displayProfile(), canModerate(), display(), and getType(). This prevents incomplete objects from being instantiated.

## 4.7 File Handling

The FileManager class centralizes all file I/O using the following standard C++ streams:

- std::ofstream — writing users and chat files; opened with default truncation mode to overwrite
- std::ifstream — reading users and chat files line by line
- std::ios::app — used in Logger<T>::log() to append entries without overwriting the log
- std::filesystem (C++17) — create\_directories() ensures data/ and data/private\_chats/ exist; fs::exists() checks before loading; directory\_iterator() lists all chat files when searching

Example — saving users with encrypted password:

```
// FileManager.cpp
std::ofstream file(usersFile_);
EncryptionManager em;
for (const auto& u : users) {
    std::string encPwd = em.encrypt(u->getPassword());
    file << getUserType(u.get()) << "," << u->getUsername()
        << "," << encPwd << ",...\n";
}
```

Chat files follow a simple line-based protocol: the first line is "user1,user2"; each subsequent line is "Type,Sender,Content,Timestamp,IsRead".

## 4.8 Static Members

The User class declares a static constant integer to store the maximum number of allowed login attempts:

```
// User.h
static const int kMaxLoginAttempts = 3;
```

Being static, this constant is shared across all User instances and is evaluated at compile time. EncryptionManager similarly uses a static constant kDefaultShift = 3 for the default Caesar shift value.

## 4.9 Templates

The Logger class is a class template parameterized on the message type T:

```
// Logger.h
template <typename T>
class Logger {
```



```

private:
    std::string logFile_;
public:
    explicit Logger(const std::string& fileName);
    void log(const T& message) const;
};

```

In main.cpp, it is instantiated as `Logger<string> logger("app")`, making it a string logger. The template design allows future instantiation with other message types (e.g., custom struct) without changing the Logger code.

## 4.10 STL Containers and Algorithms

The following STL facilities are actively used:

- `std::vector<std::unique_ptr<User>>` — dynamic array of user objects in main; ownership is managed by `unique_ptr`
- `std::unordered_map<string, unique_ptr<PrivateChat>>` —  $O(1)$  average lookup of chat sessions by chat ID
- `std::set<std::string>` `blockedUsers_` — in User class; provides  $O(\log n)$  lookup for blocked usernames with automatic ordering and uniqueness
- `std::find_if` — used in main.cpp to search for a user by username in the vector before registering or removing
- `std::move` — used throughout when transferring ownership of `unique_ptr` objects into containers
- `std::vector<const Message*>` — non-owning result set returned by `SearchEngine::search()`

## 4.11 Exception Handling

All critical operations are wrapped in try-catch blocks. Constructors of User and Message throw `std::invalid_argument` if inputs are empty. FileManager operations throw `std::runtime_error` if files cannot be opened. In main.cpp, registration, message sending, and password reset are all wrapped:

```

// main.cpp - message sending
try {
    if (content.empty())
        throw std::invalid_argument("Message content cannot be empty.");
    auto msg = make_unique<EncryptedMessage>(currentUser->getUsername(),
content);
    chat->addMessage(move(msg));
    fm.savePrivateChat(chat);
} catch (const std::exception& e) {
    printError("Error: " + string(e.what()));
}

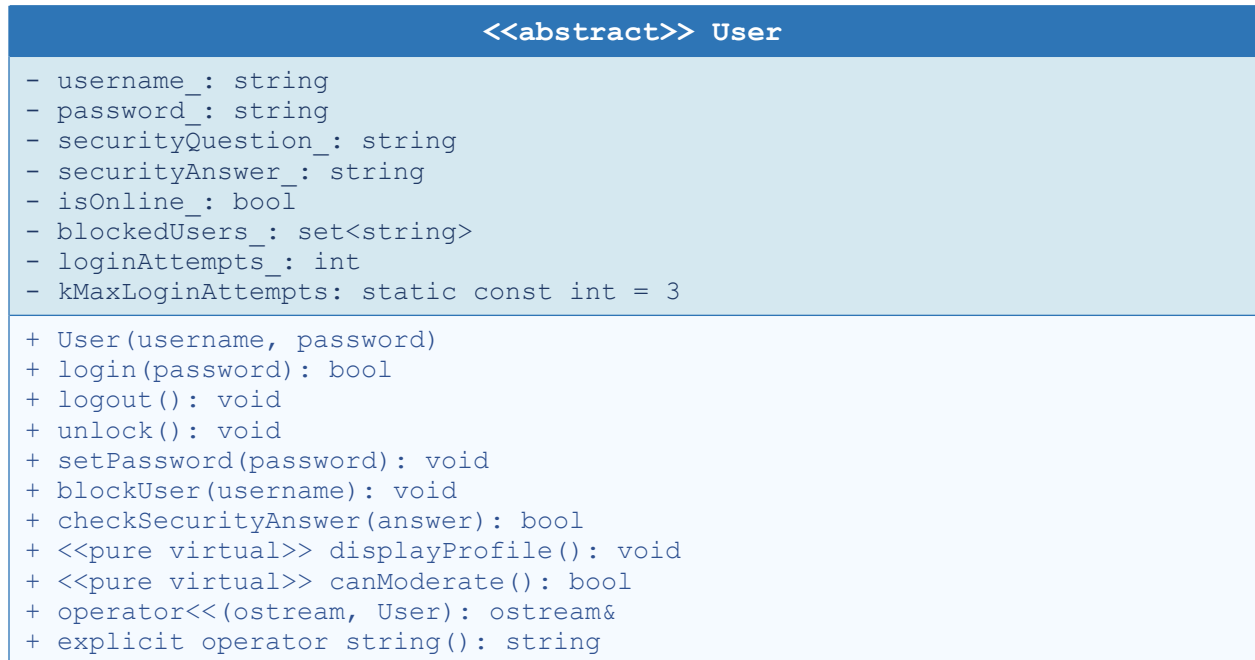
```

---

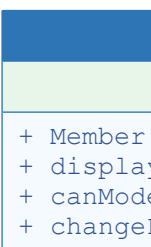
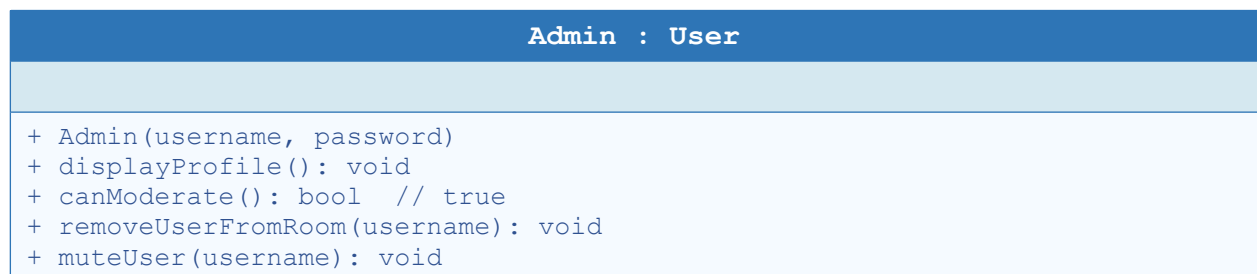
## 5. System Design (UML Class Diagram)

The following text-based UML class diagrams show each class with its attributes, methods, and relationships. Arrows show inheritance (is-a) and association (uses/owns).

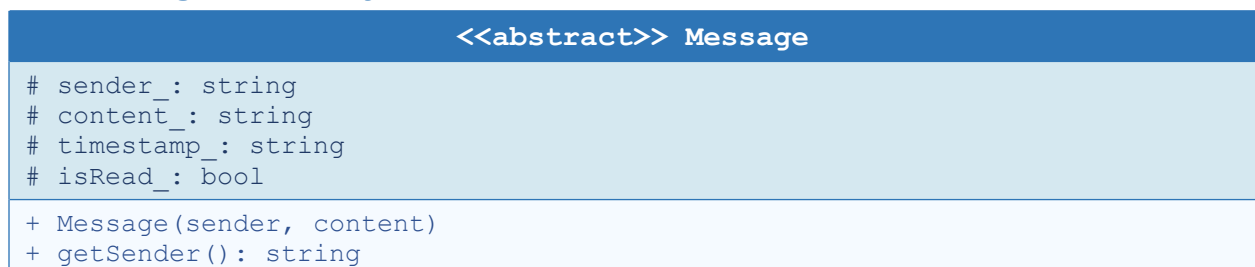
### 5.1 User Hierarchy



▲ inherits ▲



### 5.2 Message Hierarchy



```

+ getTimestamp(): string
+ setIsRead(bool): void
+ <<pure virtual>> display(): void
+ <<pure virtual>> getType(): string
+ virtual getContent(): string
+ operator<<(ostream, Message): ostream&

```

▲ inherits ▲

| TextMessage : Message                                                                       |
|---------------------------------------------------------------------------------------------|
| <pre> + TextMessage(sender, content) + display(): void + getType(): string // "Text" </pre> |

|                                                      |
|------------------------------------------------------|
| <pre> - shift_ - encrypt </pre>                      |
| <pre> + Encrypt + display + getType + getCont </pre> |

## 5.3 Supporting Classes

| PrivateChat                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> - id_: string - user1_: string - user2_: string - messages_: vector&lt;unique_ptr&lt;Message&gt;&gt; </pre>                                                                                                                               |
| <pre> + PrivateChat(user1, user2) + addMessage(unique_ptr&lt;Message&gt;): void + viewHistory(): void + markAsRead(): void + getUnreadCount(): int + getId(): string + getMessages(): const vector&lt;unique_ptr&lt;Message&gt;&gt;&amp; </pre> |

| FileManager                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> - usersFile_: string - privateChatsDir_: string </pre>                                                                                                                                                                                                                                                                                              |
| <pre> + FileManager() + saveUsers(vector&lt;unique_ptr&lt;User&gt;&gt;): void + loadUsers(): vector&lt;unique_ptr&lt;User&gt;&gt; + savePrivateChat/PrivateChat*: void + loadPrivateChat(id): unique_ptr&lt;PrivateChat&gt; + privateChatExists(id): bool + deletePrivateChatFile(id): void + getAllChatIdsForUser(username): vector&lt;string&gt; </pre> |

| SearchEngine |
|--------------|
|              |

```
+ search(messages, keyword): vector<const Message*>
+ search(messages, keyword, sender): vector<const Message*> // overloaded
```

#### EncryptionManager

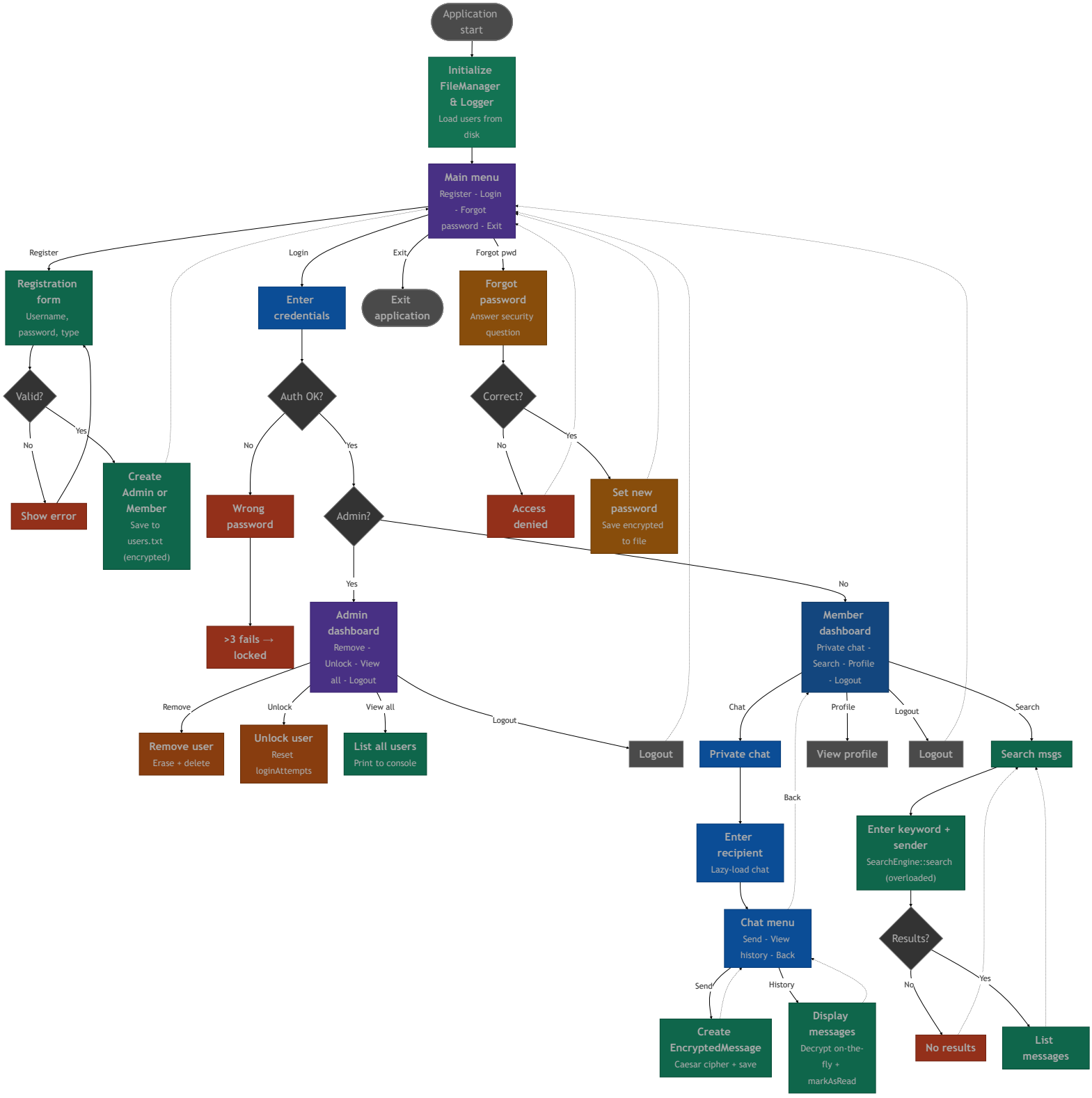
```
- kDefaultShift: static const int = 3
+ encrypt(text, shift=3): string
+ decrypt(text, shift=3): string
```

#### Logger<T>

```
- logFile_: string
+ Logger(fileName: string)
+ log(message: T): void
```

## 5.4 Relationships Summary

- Admin and Member both inherit from User (is-a relationship)
  - TextMessage and EncryptedMessage both inherit from Message (is-a relationship)
  - PrivateChat owns Message objects via vector<unique\_ptr<Message>> (composition)
  - EncryptedMessage composes EncryptionManager (composition for encrypt/decrypt)
  - FileManager uses User, PrivateChat, Message, EncryptionManager (association)
  - main() associates with all classes through object instantiation and pointer references
-



## 6. Key Logic and Implementation

### 6.1 Memory Management

The project adopts modern C++ smart pointer idioms throughout to eliminate memory leaks:

- `std::unique_ptr<User>` — every user object is heap-allocated via `make_unique<Admin>()` or `make_unique<Member>()` and stored in a vector. Ownership is transferred using `std::move()` when pushed into the vector. When a user is erased from the vector, the `unique_ptr` destructor automatically frees the memory.
- `std::unique_ptr<PrivateChat>` — each chat session is heap-allocated and stored in an `unordered_map`. Lazy loading is used: a chat is only loaded from disk when first accessed.
- `std::unique_ptr<Message>` — each message is heap-allocated via `make_unique<EncryptedMessage>()` and moved into `PrivateChat::messages_`. The `PrivateChat` destructor automatically frees all message memory.
- Raw pointer `User* currentUser` — used as a non-owning observer pointer to the currently logged-in user. The object it points to is owned by the users vector; `currentUser` is set to `nullptr` on logout.

No explicit delete calls appear anywhere in the codebase — all heap memory is exclusively managed through smart pointers.

### 6.2 File I/O Design

The file I/O design follows a simple, human-readable flat-file format:

**users.txt format:**

```
Admin,alice,encryptedPwd,What is your pet's name?,encryptedAnswer
Member,bob,encryptedPwd,What city were you born in?,encryptedAnswer
```

**private\_chats/<user1\_user2>.txt format:**

```
alice,bob
Encrypted,alice,hqoor,Thu May 1 10:00:00 2025,1
Encrypted,bob,jrrg pruglqj,Thu May 1 10:01:00 2025,0
```

When loading, `FileManager` reads the first line to reconstruct the `PrivateChat` object, then parses subsequent lines to recreate message objects. The type field determines whether to instantiate `TextMessage` or `EncryptedMessage`. Encrypted content is decrypted before being passed to the `EncryptedMessage` constructor (which will re-encrypt it for in-memory storage).

### 6.3 Program Flow (Menu-Driven Logic)

The `main()` function runs an infinite `while(true)` loop. The branch taken depends on whether `currentUser` is `nullptr`:

- `currentUser == nullptr`: Show the login menu (Register / Login / Forgot Password / Exit)
- `currentUser != nullptr && canModerate() == true`: Show the Admin menu
- `currentUser != nullptr && canModerate() == false`: Show the Member/Dashboard menu

Menu choices are read as integers. Invalid non-numeric input is handled by checking the return value of `cin >> choice` and calling `clearInput()` to flush the stream. Each operation is followed by `printSuccess()` or `printError()` with a 1.5-second delay, then the screen is cleared for the next iteration.

---

## 7. Testing and Results

### 7.1 Test Case 1 — Successful User Registration

**Input:** Username: alice, Password: pass123, Type: member

**Security Q:** What is your pet's name? / Answer: fluffy

**Expected Output:** "Registration successful! You may now log in."

**File Effect:** data/users.txt now contains a Member record with encrypted password and answer

### 7.2 Test Case 2 — Duplicate Username Registration

**Input:** Username: alice (already registered)

**Expected Output:** "Error: Username already exists! Please choose another."

**File Effect:** No change to users.txt

### 7.3 Test Case 3 — Login with Correct Credentials

**Input:** Username: alice, Password: pass123

**Expected Output:** "Login successful! Welcome, alice." — Member dashboard displayed

### 7.4 Test Case 4 — Account Lockout after 3 Failed Logins

**Input:** Wrong password entered 3 times for username: alice

**Expected Output:** "Too many failed attempts. Account locked." on the 3rd attempt

**Resolution:** Admin logs in and selects "Unlock User" -> enters "alice" -> account unlocked

### 7.5 Test Case 5 — Sending and Viewing an Encrypted Message

**Input:** alice chats with bob; sends message: "hello world"

**Expected Output:** "Message sent." — chat saved; on viewing: "[timestamp] alice: hello world (Encrypted)"

**File Content:** Encrypted,alice,khoor zruog,<timestamp>,0 (Caesar shift of 3)

### 7.6 Test Case 6 — Message Search

**Input:** Keyword: "hello", Sender filter: "alice"

**Expected Output:** Lists the message "hello world" with timestamp and sender

**Input (no results):** Keyword: "xyz123"

**Expected Output:** "No messages found matching your criteria."

### 7.7 Test Case 7 — Password Reset via Security Question

**Input:** Username: alice, correct answer: fluffy, new password: newpass456

**Expected Output:** "Password reset successful! You can now log in."

### 7.8 Test Case 8 — Admin Removing a User

**Input:** Admin logs in; selects Remove User; enters: bob

**Expected Output:** "User removed successfully!" — bob's record deleted from users.txt; bob's chat files deleted

# Test Cases For Registration feature

## 1.Cant keep the username or password feild empty

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice:
1
> Enter Username:
> Enter Password:
> Enter Type (admin/member): member
Error: Username and password cannot be empty.
```

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 1
> Enter Username: noorejannat
> Enter Password: 1234
> Enter Type (admin/member): member

--- Security Question Setup ---
> Set a security question (e.g. What is your pet's name?): what's your fav color?
> Set the answer: blue
Loading...

Registration successful! You may now log in.
```

## 2.two users with same username cant exist

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 1
> Enter Username: noorejannat
> Enter Password: 5678
> Enter Type (admin/member): member
Error: Username already exists! Please choose another.
```

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 1
> Enter Username: noorejannat
> Enter Password: 1234
> Enter Type (admin/member): member

--- Security Question Setup ---
> Set a security question (e.g. What is your pet's name?): what's your fav color?
> Set the answer: blue
Loading...

Registration successful! You may now log in.
```

## 3.you cant keep the Security Question field empty

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 1
> Enter Username: irfan_sadik
> Enter Password: 1234
> Enter Type (admin/member): member

--- Security Question Setup ---
> Set a security question (e.g. What is your pet's name?):
> Set the answer:
Error: Security question and answer cannot be empty.
```

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 1
> Enter Username: irfan_sadik
> Enter Password: 1234
> Enter Type (admin/member): member\

--- Security Question Setup ---
> Set a security question (e.g. What is your pet's name?): what's your fav colour?
> Set the answer: blue
Loading...

Registration successful! You may now log in.
```



# Login Test Cases with Sample Inputs

1.user cant login with wrong password

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 2
> Enter Username: noorejannat
> Enter Password: 55555
Loading...

Error: Invalid password!
```

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 2
> Enter Username: noorejannat
> Enter Password: 1234
Loading...

Login successful! Welcome, noorejannat.
```

2.If some one tries to enter invalid password more than 3 times account gets locked,to unlock it the user have to reset the password from forgot password option

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 2
> Enter Username: noorejannat
> Enter Password: 4444
Loading...

Error: Invalid password!
```

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 2
> Enter Username: noorejannat
> Enter Password: 55555
Loading...

Too many failed attempts. Account locked.
Error: Invalid password!
```

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 3
> Enter your username: noorejannat
Security Question: what's your fav color?
> Your Answer: blue
> Answer correct! Enter new password: 230406
> Confirm new password: 230406
Loading...

Password reset successful! You can now log in.
```

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 2
> Enter Username: noorejannat
> Enter Password: 55555
Loading...

Error: Invalid password!
```

```
D:\Projects\cpp_projects\kilo_ x + v

=====
CHAT APPLICATION
=====

[1] Register
[2] Login
[3] Forgot Password
[4] Exit

-----

> Enter your choice: 2
> Enter Username: noorejannat
> Enter Password: 230406
Loading...

Login successful! Welcome, noorejannat.
```

## Password Recovery Test cases

1.if user gives wrong answer,password recovery gets denied

```
D:\Projects\cpp_projects\kilo. x + v
=====
CHAT APPLICATION
=====
[1] Register
[2] Login
[3] Forgot Password
[4] Exit

> Enter your choice: 3
> Enter your username: irfan_sadik
Security Question: what's your fav colour?
> Your Answer: red
Error: Incorrect answer! Password reset denied.
```

2.If new passwords dont match system will throw error

```
D:\Projects\cpp_projects\kilo. x + v
=====
CHAT APPLICATION
=====
[1] Register
[2] Login
[3] Forgot Password
[4] Exit

> Enter your choice: 3
> Enter your username: irfan_sadik
Security Question: what's your fav colour?
> Your Answer: blue
> Answer correct! Enter new password: 6767
> Confirm new password: 5656
Error: Passwords do not match!
```

3.user cant keep new password empty

```
D:\Projects\cpp_projects\kilo. x + v
=====
CHAT APPLICATION
=====
[1] Register
[2] Login
[3] Forgot Password
[4] Exit

> Enter your choice: 3
> Enter your username: irfan_sadik
Security Question: what's your fav colour?
> Your Answer: blue
> Answer correct! Enter new password:
> Confirm new password:
Error: Password cannot be empty.
```

correct way to reset password

```
D:\Projects\cpp_projects\kilo. x + v
=====
CHAT APPLICATION
=====
[1] Register
[2] Login
[3] Forgot Password
[4] Exit

> Enter your choice: 3
> Enter your username: irfan_sadik
Security Question: what's your fav colour?
> Your Answer: blue
> Answer correct! Enter new password: 6767
> Confirm new password: 6767
Loading...

Password reset successful! You can now log in.
```

# Test Cases For Private Chat

## 1. User Can't Chat with User's account

```
D:\Projects\cpp_projects\kilo. x + v
=====
DASHBOARD
=====
[1] Private Chat
[2] Search Messages
[3] Profile
[4] Logout

-----
> Enter your choice: 1
> Enter recipient username: noorejannat
Error: You cannot chat with yourself.
```

## 2. User can't send text to a non-existing user

```
D:\Projects\cpp_projects\kilo. x + v
=====
DASHBOARD
=====
[1] Private Chat
[2] Search Messages
[3] Profile
[4] Logout

-----
> Enter your choice: 1
> Enter recipient username: sadman
Error: User not found!
```

## 3. User can't send empty texts

```
=====
CHAT: irfan_sadik
=====
[1] Send Message
[2] View Chat History
[3] Back

-----
> Enter your choice: 1
> Enter message:
Error: Message content cannot be empty.
```

## 4. User can't send special character as text

```
=====
CHAT: irfan_sadik
=====
[1] Send Message
[2] View Chat History
[3] Back

-----
> Enter your choice: 1
> Enter message: #^&!@
Loading...

Error: Message content cannot be empty.
```

if texts are sent properly user can see them in chat history and also gets encrypted in file

```
=====
CHAT HISTORY
=====
[Mon May 4 16:21:26 2026] noorejannat: hello irfan kemon acho (Encrypted)
[Mon May 4 16:22:40 2026] noorejannat: hello irfan (Encrypted)

Press Enter to go back...|
```

```
noorejannat,irfan_sadik
Encrypted,noorejannat,khoor luidq nhprq dfkr,Mon May 4 16:21:26 2026,1
Encrypted,noorejannat,khoor luidq,Mon May 4 16:22:40 2026,1
```

## Test Cases for serch feature

1.user cant keep the input feild blan

```
=====
DASHBOARD
=====
[1] Private Chat
[2] Search Messages
[3] Profile
[4] Logout

> Enter your choice: 2
> Enter keyword to search:
> Enter sender username to filter by (or leave blank to search all):
Error: Keyword cannot be empty.
```

2.if there is no message found it returns nothing\

```
=====
SEARCH RESULTS (0)
=====
No messages found matching your criteria.
Press Enter to go back...|
```

if it finds messages relevant to the keywords it returns the text messages

```
=====
SEARCH RESULTS (2)
=====
[Mon May 4 16:21:26 2026] noorejannat: hello irfan kemon acho (Encry
[Mon May 4 16:22:40 2026] noorejannat: hello irfan (Encrypted)
Press Enter to go back...
```

```
=====
DASHBOARD
=====
[1] Private Chat
[2] Search Messages
[3] Profile
[4] Logout

> Enter your choice: 2
> Enter keyword to search: hello
> Enter sender username to filter by (or leave blank to search all):
Loading.|
```

## Test Cases For Admin Feature

### 1. admin cant remove themselves

```
=====
ADMIN MENU
=====

[1] Remove User
[2] Unlock User
[3] View All Users
[4] Logout

-----
> Enter your choice: 1
> Enter username to remove: mawajannat
Error: You cannot remove yourself.
|
```

2.if admin removes a user it also got removed from the database and their chats get deleted also

```
=====
ADMIN MENU
=====

[1] Remove User
[2] Unlock User
[3] View All Users
[4] Logout

-----
> Enter your choice: 1
> Enter username to remove: irfan_sadik
User removed successfully!
```

|                                                                                                    |                  |               |      |
|----------------------------------------------------------------------------------------------------|------------------|---------------|------|
|  afrozabegum_noor | 5/1/2026 4:19 PM | Text Document | 0 KB |
|  irfan_noor       | 5/1/2026 4:22 PM | Text Document | 1 KB |

## 8. Challenges and Debugging

### 8.1 Incorrect File Parsing with Commas in Messages

Challenge: Messages containing commas broke the CSV parsing in `FileManager::loadPrivateChat()` because `std::getline(ss, field, ',')` would split on a comma inside the message content.

Solution: The message content field is guaranteed to be the third field, and the remaining fields (timestamp, isRead) are the last two. The parsing was structured to read type, sender, content, then timestamp and isRead in strict order, relying on the fact that timestamps from `ctime()` never contain commas.

### 8.2 Double-Encryption of Messages on Reload

Challenge: When a saved `EncryptedMessage` was loaded from disk (already encrypted), passing it directly to the `EncryptedMessage` constructor resulted in double encryption.

Solution: In `FileManager::loadPrivateChat()`, the saved content is first decrypted using `EncryptionManager::decrypt()` before being passed to the `EncryptedMessage` constructor, which re-encrypts it for in-memory storage. This ensures a single layer of encryption at all times.

### 8.3 `std::cin` Stream in Bad State After Non-Numeric Input

Challenge: If a user typed a letter instead of a menu number, `cin` would enter a fail state and all subsequent reads would silently fail, causing an infinite loop.

Solution: The menu read was changed to check if `!(cin >> choice)` and call `clearInput()` (which calls `cin.clear()` followed by `cin.ignore(numeric_limits<streamsize>::max(), '\n')`) to fully flush the error state and the bad input.

### 8.4 Inconsistent Private Chat ID Generation

Challenge: If alice starts a chat with bob, the ID is "alice\_bob". If bob starts a chat with alice, the code would generate "bob\_alice", creating a second file instead of reusing the existing one.

Solution: Both `PrivateChat::PrivateChat()` and the chat-creation logic in `main.cpp` sort the two usernames alphabetically before concatenating, ensuring the ID is always in the same canonical order regardless of who initiates the chat.

### 8.5 Filesystem Directory Creation Errors on First Run

Challenge: On first launch, the `data/` and `data/private_chats/` directories do not exist, causing file open operations to silently fail.

Solution: The `FileManager` constructor calls `std::filesystem::create_directories()` for all required directories inside a try-catch block. This is executed before any file I/O, guaranteeing the directories exist.

---

## 9. Limitations

### 9.1 Unimplemented Features

- Group chat: The application only supports one-to-one private messaging. A group chat room with multiple participants is not implemented.
- Message deletion: Users cannot delete individual messages; only an admin can delete all chats associated with a removed user.
- Profile editing: Users cannot change their username or security question after registration.
- Block list enforcement: While the User class maintains a blockedUsers\_ set and blockUser()/unblockUser() methods, the messaging logic in main.cpp does not currently check the block list before allowing a message to be sent.

### 9.2 Performance Limitations

- The entire users vector is rewritten to disk on every save operation. For large user bases, this is inefficient; a database or indexed file would perform better.
- Chat history is loaded entirely into memory when a chat session is opened. For very long conversations, this could consume significant RAM.

### 9.3 Security Limitations

- The Caesar cipher (shift of 3) used for password and message encryption is a very weak algorithm. It provides obfuscation, not real security. Production systems require bcrypt or AES.
- Passwords are stored in a plaintext-derived Caesar-encrypted format. A salt-based hash function would be the appropriate solution.

### 9.4 Platform Limitations

- The application uses system("cls") (Windows-only) and ANSI color codes. On Linux/macOS, system("clear") would be needed and ANSI support depends on the terminal.
  - The interface is purely console-based with no GUI, which limits accessibility and usability for non-technical users.
-

## 10. Conclusion

This project successfully demonstrates the practical application of Object-Oriented Programming principles in building a real-world, file-backed chat application entirely in C++17. The use of abstract base classes (User, Message) with pure virtual methods enforced a clear contract between the system's components and enabled runtime polymorphism — the mechanism that drives the admin/member role split and the transparent message encryption/decryption.

Encapsulation ensured that each class owns and controls its data. Inheritance eliminated code duplication by placing shared attributes (username, password, online status) in the User base class while letting Admin and Member specialize their behavior. Composition was used in PrivateChat and EncryptedMessage, and function overloading in SearchEngine demonstrated compile-time polymorphism.

Modern C++ features — `std::unique_ptr`, `std::filesystem`, `std::unordered_map`, and `std::move` — made the implementation memory-safe and expressive. The template-based `Logger<T>` class showed how generic programming can produce reusable utilities without code duplication.

### 10.1 Future Scope

- GUI Version: Replace the console interface with a Qt or wxWidgets graphical user interface for a more user-friendly experience.
  - Database Integration: Replace flat-file storage with SQLite or PostgreSQL to enable indexed search, concurrent access, and efficient querying over large datasets.
  - Networking: Implement real-time messaging using BSD sockets or the Boost.Asio library to enable communication between users on different machines.
  - Strong Encryption: Replace the Caesar cipher with AES-256 (via OpenSSL) and use bcrypt for password hashing.
  - Multimedia Messaging: Extend the Message hierarchy with ImageMessage and FileAttachmentMessage subclasses.
  - Group Chat: Add a GroupChat class that holds multiple participants and broadcasts messages to all members.
-



## 11. References

- [cplusplus.com](https://cplusplus.com) — C++ Standard Library Reference: <https://cplusplus.com>
- [cppreference.com](https://en.cppreference.com) — Comprehensive C++ Language and Library Reference: <https://en.cppreference.com>
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
- Lippman, S., Lajoie, J., & Moo, B. (2012). C++ Primer (5th ed.). Addison-Wesley.
- ISO/IEC 14882:2017 — Programming Language C++ (C++17 Standard)
- Microsoft Docs — `std::filesystem`: <https://docs.microsoft.com/en-us/cpp/standard-library/filesystem>
- GNU GCC Documentation: <https://gcc.gnu.org/onlinedocs/>